Let's talk a bit about the `respond` function in the `CH8-app.py` file – see *Figure 8.34*, which is passed to our app's UI chatbot. The `respond` function, in this case, is the caller of your deployed endpoints, where we do not just send and receive responses but also can shape the format of the input or output. In our case, the endpoint expects to receive a request in the format of a JSON with field inputs with the questions within a list, while the output is a JSON with field predictions.

```
import itertools
import gradio as gr
import requests
import os
from gradio.themes.utils import sizes


def respond(message, history):

    if len(message.strip()) == 0:
        return "ERROR the question should not be empty"

    # Set your secrets under HF secrets for your Space
    local_token = os.getenv('API_TOKEN')
    local_endpoint = os.getenv('API_ENDPOINT')

    if local_token is None or local_endpoint is None:
        return "ERROR missing env variables"

    # Add your API token to the headers
    headers = {
        'Content-Type': 'application/json',
        'Authorization': f'Bearer {local_token}'
    }

    # Pay attenction to what your model expects as schema format
    q = {"inputs": [message]}
    try:
        response = requests.post(
            local_endpoint, json=q, headers=headers, timeout=100)
        response_data = response.json()
        response_data=response_data["predictions"][0]

    except Exception as error:
        response_data = f"ERROR status_code: {type(error).__name__}"

    return response_data
```

Figure 8.34 – The respond function written in the Gradio app